

## Hashing, MACs & Data Integrity

### Hash Functions: One-Way Summaries

A cryptographic hash (SHA-256, SHA3):

Maps ANY input → fixed length output (256 bits)

**Deterministic** – same input, always same output

**Preimage-resistant** – can't reverse output→input

**Avalanche** – one changed bit changes ~half the output bits

**Collision-resistant** – hard to find two inputs with the same hash

### Uses of Hashing

| Use                 | Example                              |
|---------------------|--------------------------------------|
| Password storage    | Store hash, not plaintext            |
| File integrity      | sha256sum of a downloaded ISO        |
| Evidence integrity  | Hash the forensic image before/after |
| Git/software        | Verify code not tampered             |
| Duplicate detection | dedupe by hash                       |

### Why Password Hashing Needs More Than a Hash

A plain SHA-256 of passwords is instantly crackable: attackers precompute hashes for millions of common passwords (**rainbow tables**) and just look up. Defense:

**Salt** – random per-user value mixed into the hash → defeats rainbow tables

**Slow hash** – bcrypt/argon2/scrypt with adjustable "cost" → defeats GPU brute force (each guess costs the attacker time)

```
import bcrypt
h = bcrypt.hashpw(b"correct horse battery", bcrypt.gensalt(rounds=10))
print(h) # $2b$10$<salt>$<hash>
print(bcrypt.checkpw(b"wrong", h)) # False
```

## HMAC: Authenticating Integrity

A plain hash proves "data didn't change" but not "data is from the right person". An **HMAC** (Hash-based Message Authentication Code) is a hash computed with a secret key. Only someone with the key can produce a valid HMAC.

```
import hmac, hashlib
mac = hmac.new(b"shared-secret", b"message", hashlib.sha256).hexdigest()
```

HMACs verify integrity + authenticity (with shared secret). Used in API signatures, token validation, TLS.

## Real-World Integrity Stories

**SolarWinds (2020)** – attackers signed malicious update; integrity check passed because the signature was genuinely theirs... after they stole the signing key. Key protection matters as much as the algorithm.

**WannaCry (2017)** – the disaster was less the crypto, more the unpatched SMB. Crypto didn't save it because the flaw was elsewhere.

**Fake Microsoft installers** – signed with stolen codesigning certs.

**Takeaway:** integrity tooling (hashes, signatures) fails when the *trust anchor* (the key) is compromised. Protecting keys is a security discipline of its own.